

Exercício Prático: API de Catálogo de Filmes

Turma: LionsDev **Tópicos:** APIs REST com Express.js, CRUD, JSON, arrays de objetos, parâmetros de rota, query params, validação de dados e status codes HTTP.

1. Contexto

Um pequeno clube de cinema quer criar uma API para organizar os filmes exibidos nas sessões. A primeira versão do sistema deve permitir cadastrar filmes, listar o catálogo, atualizar dados, remover registros e buscar filmes por filtros.

Nesta etapa, o sistema deve usar apenas um array em memória, sem banco de dados.

2. Configuração Inicial

Crie uma API Node.js com Express.

Requisitos iniciais:

- O arquivo principal deve se chamar `index.js`.
- A API deve rodar na porta `3000`.
- Use `express.json()` para permitir requisições com corpo em JSON.
- Crie um array chamado `filmes`.
- Crie uma variável `proximoId` iniciando em `1`.

Cada filme deve seguir esta estrutura:

```
{
  id: 1,
  titulo: "Cidade de Deus",
  diretor: "Fernando Meirelles",
  ano: 2002,
  genero: "Drama"
}
```

3. Requisitos Obrigatórios

3.1 Cadastrar filme (CREATE)

Crie a rota **POST /filmes**.

O corpo da requisição deve receber:

```
{
  "titulo": "Cidade de Deus",
  "diretor": "Fernando Meirelles",
  "ano": 2002,
  "genero": "Drama"
}
```

Regras:

- Todos os campos (**titulo** , **diretor** , **ano** , **genero**) são obrigatórios.
- Se algum campo estiver faltando, responda com status **400** e uma mensagem de erro.
- O **id** deve ser gerado automaticamente usando **proximoId**.
- O novo filme deve ser salvo no array **filmes**.
- Ao cadastrar com sucesso, retorne o filme criado com status **201**.

3.2 Listar filmes (READ)

Crie a rota **GET /filmes**.

Regras:

- A rota deve retornar todos os filmes cadastrados.
- Se o catálogo estiver vazio, retorne um array vazio **[]**.
- Use status **200**.

3.3 Atualizar filme (UPDATE)

Crie a rota **PUT /filmes/:id**.

O corpo da requisição pode receber apenas os dados que devem ser alterados:

```
{
  "diretor": "Fernando Meirelles e Kátia Lund",
  "genero": "Drama Policial"
}
```

Regras:

- Busque o filme pelo `id` recebido em `req.params`.
- Converta o `id` para número antes de comparar.
- Se o filme não existir, responda com status `404` e uma mensagem de erro.
- Atualize apenas os campos enviados: `titulo`, `diretor`, `ano` ou `genero`.
- Retorne o filme atualizado com status `200`.

3.4 Deletar filme (DELETE)

Crie a rota `DELETE /filmes/:id`.

Regras:

- Localize o índice do filme usando `.findIndex()`.
- Se o filme não existir, responda com status `404`.
- Remova o filme do array usando `.splice()`.
- Retorne uma mensagem de sucesso com status `200`.

3.5 Buscar filmes (QUERY PARAMS)

Crie a rota `GET /filmes/busca`.

A busca deve aceitar os seguintes filtros opcionais:

- `titulo`
- `diretor`
- `ano`
- `genero`

Exemplos de URLs:

```
http://localhost:3000/filmes/busca?titulo=cidade  
http://localhost:3000/filmes/busca?diretor=fernando  
http://localhost:3000/filmes/busca?ano=2002  
http://localhost:3000/filmes/busca?genero=drama
```

Regras:

- `titulo`, `diretor` e `genero` devem aceitar parte do texto e ignorar letras maiúsculas/minúsculas.

- `ano` deve ser comparado como número.
 - Se mais de um filtro for enviado, aplique todos os filtros em sequência.
 - Retorne os resultados encontrados com status `200`.
-

4. Testes Esperados

Teste a API no Postman.

Sugestões de testes:

1. Cadastre pelo menos 3 filmes.
 2. Liste todos os filmes.
 3. Atualize apenas o gênero de um filme.
 4. Atualize apenas o diretor de um filme.
 5. Tente atualizar um `id` inexistente.
 6. Delete um filme cadastrado.
 7. Faça buscas por título, diretor, ano e gênero.
-

5. Dicas para a Implementação

- `req.body` contém os dados enviados no corpo da requisição.
 - `req.params` contém valores obrigatórios que aparecem na rota.
 - `req.query` contém filtros opcionais enviados na URL.
 - Use `parseInt()` para converter `id` e `ano` quando necessário.
 - Para buscas por texto, combine `.toLowerCase()` com `.includes()`.
 - Mantenha as respostas da API claras, sempre com status HTTP adequado.
-

LionsDev • Professor Nicolas Cardoso Motta

Exercício Prático de APIs REST com Express.js - Módulo 07